PROJECT THESIS REPORT

Sentinel-LLM

LoRA Fine-Tuning for Cybersecurity Threat Intelligence: A From-Scratch Implementation, a Controlled Efficiency Study, and a Production QLoRA Path

HEADLINE RESULTS

100%
LoRA test accuracy
191/191, = full fine-tuning

1.86%
Trainable parameters
10,672 of 574,923

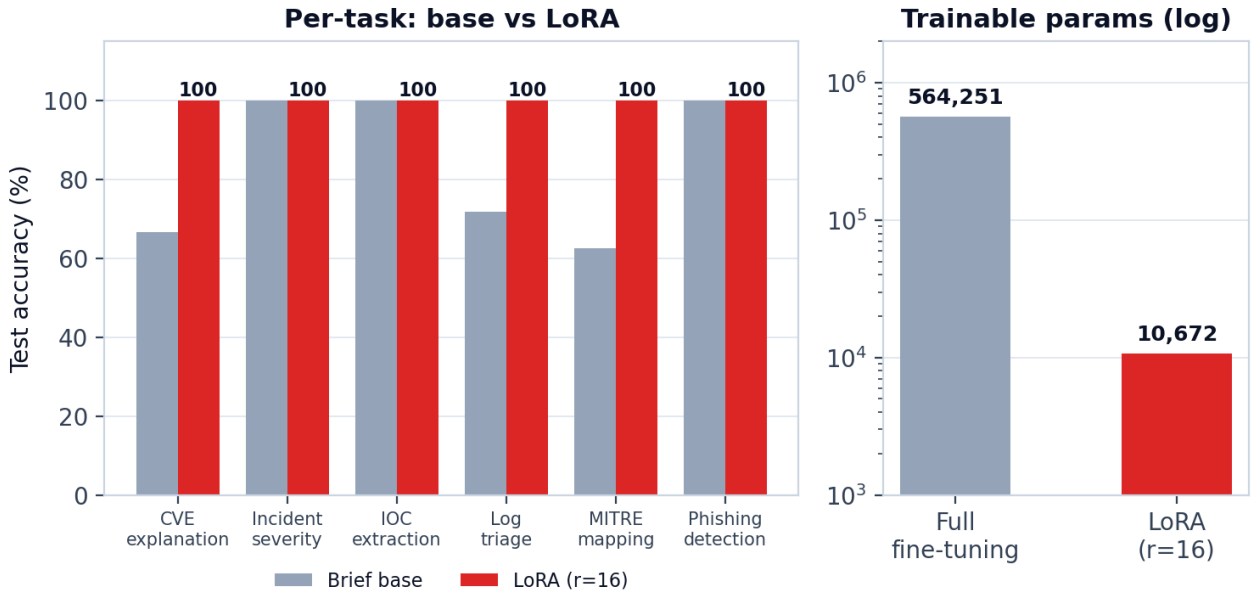
52.9x
Fewer trainable params
than full fine-tuning

83.25% > 100%
Base > LoRA accuracy
+16.75 points

Ashutosh Sahoo
M.Tech, Artificial Intelligence · NIT Silchar
November 2026

Results at a Glance

100% LoRA test accuracy 191/191, = full fine-tuning	**1.86%** Trainable parameters 10,672 of 574,923
52.9x Fewer trainable params than full fine-tuning	**83.25% > 100%** Base > LoRA accuracy +16.75 points



Left: per-task accuracy, brief base model vs LoRA — adaptation matters exactly on MITRE mapping, CVE explanation and log triage. Right: trainable parameters, full fine-tuning vs LoRA (log scale).

Contents

Abstract

1. Introduction

2. Background and Theory

3. System Design and Architecture

4. Implementation -- A File-by-File, Function-by-Function Walkthrough

5. Dataset

6. Experiments

7. Results and Analysis

8. Discussion, Limitations and the Production Path

9. Conclusion and Future Work

10. Learning Guide

11. How to Run

12. References

Appendix A -- Full results.json

Appendix B -- File Listing

Abstract

Security operations centres (SOCs) process a continuous stream of authentication logs, firewall and intrusion-detection alerts, phishing emails, threat-intelligence reports, vulnerability (CVE) disclosures and incident tickets. Each item must be classified, mapped to a shared adversary-behaviour vocabulary such as MITRE ATT&CK, stripped of indicators of compromise (IOCs), explained to junior analysts, or assigned a severity -- quickly, consistently, and in a format a tired analyst at 3 a.m. can act on. Large language models (LLMs) are natural candidates for this work, but a general-purpose LLM does not know an organisation's label conventions, severity rubric or reporting style, and adapting one by full fine-tuning is expensive: every parameter is updated, every optimiser state is stored, and every task specialisation becomes another full-size copy of the model. Low-Rank Adaptation (LoRA) changed that economics. LoRA freezes the pretrained weights and learns only a small low-rank update, typically well under 5% of the parameters, yet routinely matches full fine-tuning -- which is why LoRA and its quantised variant QLoRA are now the state-of-the-art default for specialising LLMs.

This report presents Sentinel-LLM, a project that studies, implements and measures that idea end to end for cybersecurity threat intelligence. The project has two honest halves. The first is a fully runnable local system: a synthetic-but-realistic instruction dataset of 1,200 examples across six SOC tasks and 27 labels (dataset generator seed 42), a small hashed-embedding classifier in PyTorch, and -- the technical core -- LoRA implemented from scratch, including the low-rank layer, B-matrix zero initialisation, alpha/r scaling, model injection, freezing and parameter accounting. On top of that implementation sits a controlled experimental protocol modelled on how a real LLM study is run: a stratified 70/15/15 train/validation/test split (829/180/191 examples, split seed 123), a briefly trained shared base checkpoint (1 epoch, 83.25% test accuracy), a full fine-tuning baseline (8 further epochs, 564,251 trainable parameters, 100.00% test accuracy), and a four-configuration LoRA hyperparameter sweep selected on validation data only. The second half is a production script, train_lora_peft.py, that QLoRA-fine-tunes a real instruction LLM (Qwen2.5-0.5B-Instruct) on the identical dataset using 4-bit NF4 quantisation, PEFT and TRL; it is explained line by line, and no accuracy is claimed for it because it requires a GPU and has not been run.

The measured findings are threefold. First, the sweep saturates: all four configurations reach 99.44% validation accuracy (179 of 180 examples), so on this dataset rank buys confidence, not accuracy -- validation loss falls monotonically with rank, from 0.0475 at r=4 to 0.0230 at r=8 to 0.0065 at r=16, and at fixed rank the higher learning rate (8e-3) clearly beats the lower one (3e-3). Second, the selected configuration (r=16, alpha=32, learning rate 8e-3) reaches 100.00% test accuracy with 10,672 trainable parameters -- 1.86% of the model, 52.9x fewer than the full fine-tuning baseline, which reaches the same 100.00%. Third, the per-task analysis localises exactly where adaptation matters: the brief base model was already perfect on phishing detection, IOC extraction and incident severity, but scored only 62.50% on MITRE mapping, 66.67% on CVE explanation and 71.88% on log triage; LoRA lifts all three to 100.00%. The report is equally explicit about what these numbers do not show: the synthetic dataset is templated and strongly separable, so 100% demonstrates the correctness of the pipeline and the LoRA efficiency effect, not real-world SOC performance on noisy, adversarial, imbalanced production logs. Every number in the report is taken from the project's results.json, reproduced in full in Appendix A. A learning guide with a concept checklist, twelve interview questions with model answers, and extension exercises closes the report, so that a reader can study parameter-efficient fine-tuning from this document alone.

Keywords: LoRA; QLoRA; parameter-efficient fine-tuning (PEFT); large language models; instruction tuning; cybersecurity; threat intelligence; SOC automation; MITRE ATT&CK; IOC extraction; phishing

detection; hyperparameter sweep; PyTorch.

1. Introduction

1.1 The problem: a SOC analyst drowns in text

Almost everything a security operations centre touches is text. An SSH daemon writes a log line. A firewall writes another. A user forwards a suspicious email. A threat feed publishes a paragraph naming an IP address, a domain and a file hash. The National Vulnerability Database publishes a CVE description. An incident ticket arrives as two sentences typed in a hurry. Before any response can happen, each of these artefacts must pass through the same small set of judgements:

Triage. Is this log line benign, or is it a brute-force attack, a port scan, a malware beacon, a SQL injection attempt?

Mapping. Which MITRE ATT&CK technique does this observed behaviour correspond to? Shared technique IDs are how teams, tools and threat reports talk to each other.

Phishing judgement. Is this email a credential-harvesting lure or a legitimate message?

Extraction. Which exact strings in this report are indicators of compromise -- IPs, domains, hashes, URLs -- that should be blocked or hunted?

Explanation. What does this CVE actually mean, in plain language, and what is the mitigation?

Severity. Is this incident a P1 that wakes up executives, or a P4 that waits for the normal queue?

These judgements are high-volume, repetitive, convention-bound and time-critical -- precisely the profile of work that machine learning should absorb, so that human analysts spend their attention on investigation and response rather than first-pass classification. They are also judgements where the *output convention matters as much as the answer*: a severity label is only useful if it follows the organisation's rubric, and an ATT&CK mapping is only useful if it uses the correct technique ID.

1.2 Why a general LLM is not enough, and why full fine-tuning is the wrong default

A general-purpose instruction LLM can approximate all six judgements zero-shot, but three problems remain. First, **convention drift**: the model does not know this SOC's exact label set, severity boundaries or output format, and prompt engineering can only steer it so far. Second, **consistency and cost**: routing every log line through a large hosted model is expensive, slow, and sends sensitive security telemetry outside the organisation. Third, **verifiability**: an unadapted model's errors are hard to characterise because there is no held-out evaluation against the organisation's own labelled data.

The classical remedy is fine-tuning: continue training the model on labelled, task-formatted examples. But *full* fine-tuning -- updating every parameter -- carries heavy costs that grow with model size:

Memory. For each trainable parameter, a typical AdamW training loop stores the parameter itself, its gradient, and two optimiser moments. Freezing most of the model shrinks all of these proportionally.

Storage and deployment. Every fully fine-tuned task produces a complete copy of the model. Ten SOC tasks means ten full models to store, version and serve.

Catastrophic forgetting and fragility. Updating all weights on a narrow dataset can damage the model's general abilities, and small datasets make full updates noisy.

Iteration speed. Security conventions change -- new labels, new rubrics. Re-running a full fine-tune for each change is a project; re-training a small adapter is a job.

Parameter-efficient fine-tuning (PEFT) exists to remove exactly these costs, and among PEFT methods LoRA is the one that became standard practice. Sentinel-LLM is a study of that method, in that domain, with that motivation.

1.3 Objectives

The project was built against six objectives:

O1 -- Implement LoRA from first principles. No PEFT library in the local pipeline: the low-rank layer, its initialisation, scaling, injection into a model, parameter freezing and counting are all written by hand in PyTorch, so that every mechanism is visible and testable.

O2 -- Build a realistic multi-task security dataset. An instruction-formatted dataset (the same {instruction, input, output, task, label} schema used to instruction-tune real LLMs) covering six SOC tasks and 27 labels, generated with randomised entities so that the model must learn patterns rather than memorise fixed strings.

O3 -- Run a controlled efficiency study. Compare, from one shared starting checkpoint: an untrained model, a briefly trained base, a full fine-tuning baseline, and LoRA configurations -- with model selection performed on a validation split and the test split touched once per finished model.

O4 -- Characterise the hyperparameter landscape honestly. Sweep LoRA rank and learning rate, and report what actually changes (in this study: loss and confidence, not accuracy), including the tie-breaking selection rule.

O5 -- Provide the production path, clearly labelled as such. A complete QLoRA script for a real instruction LLM on GPU, explained line by line, with its un-run status stated wherever it is mentioned.

O6 -- Report with discipline. One machine-readable source of truth (results.json); per-task breakdowns; timing recorded; limitations stated next to the headline numbers, not buried.

1.4 Contributions

1. **A from-scratch LoRA implementation** (lora.py, 114 lines) with a self-test that proves the defining initialisation property: because the B matrix starts at zero, a freshly injected LoRA layer is an exact no-op, so training begins precisely from the base model's behaviour.
2. **A reproducible experimental protocol** that mirrors a real LLM adaptation study at CPU scale: shared base checkpoint, stratified splits, validation-only selection with an explicit tie-break rule, a full fine-tuning baseline, per-task metrics, and a saved checkpoint that an independent evaluation script reproduces exactly.
3. **A quantified efficiency result**: LoRA at r=16 reaches the full fine-tuning baseline's test accuracy (100.00%) with 10,672 trainable parameters against 564,251 -- 1.86% versus 100%, a 52.9x reduction -- and even the smallest swept adapter (r=4, 2,668 parameters, 0.47%) saturates validation accuracy.

4. **A per-task diagnosis** showing that adaptation gains concentrate exactly where the base model is weakest (MITRE mapping 62.50% to 100.00%; CVE explanation 66.67% to 100.00%; log triage 71.88% to 100.00%) and are zero where the base is already perfect -- the pattern a practitioner should expect when deciding where fine-tuning earns its cost.
5. **A sweep finding worth teaching**: when accuracy saturates, rank still buys calibration -- validation loss improves 7.3x from r=4 (0.0475) to r=16 (0.0065) -- so rank selection should be argued from loss and confidence behaviour, not from accuracy alone.
6. **A production QLoRA script** (train_lora_peft.py) that carries the identical dataset into a real LLM fine-tune on Kaggle/Colab-class hardware, using 4-bit NF4 with double quantisation, LoRA on all linear projections, and the model's chat template -- the standard current recipe.

1.5 Report roadmap

Chapter 2 builds the theory from first principles: fine-tuning and its costs, the low-rank hypothesis, the LoRA update rule and its initialisation, scaling, merging, QLoRA's quantisation, instruction tuning, and the evaluation concepts used later. Chapter 3 presents the system architecture and data flow. Chapter 4 walks through every file and function of the implementation, including a line-by-line reading of the GPU script. Chapter 5 describes the dataset. Chapter 6 states the full experimental protocol. Chapter 7 reports and analyses every result in results.json. Chapter 8 discusses limitations, threats to validity and the production path. Chapter 9 concludes. A learning guide, run instructions, references and appendices close the report.

2. Background and Theory

This chapter explains every concept the project depends on, in the order in which a reader needs them: what fine-tuning is and why it is expensive, why a low-rank update can be enough, the LoRA rule itself, how LoRA models are merged and served, how QLoRA adds quantisation, what instruction tuning contributes, and how classification quality is measured. Each section ends by connecting the theory to a concrete choice in Sentinel-LLM.

2.1 Fine-tuning and the cost of updating everything

A pretrained model is a function parameterised by a weight vector (in practice, a set of weight matrices). Fine-tuning continues gradient-based training of that function on task data. For a single linear layer with weight matrix W of shape (d_out, d_in), full fine-tuning learns an unrestricted update dW of the same shape: W_new = W_0 + dW, with d_in x d_out free parameters. For the local model in this project, the two MLP layers contain 128 x 256 = 32,768 and 256 x 27 = 6,912 weight parameters respectively; for a real LLM, a single attention projection can contain millions, and the model contains hundreds of such matrices.

Three costs scale with the number of *trainable* parameters, not merely with model size. Memory during training scales because gradients and AdamW's two moment buffers are stored per trainable parameter. Storage after training scales because each fully fine-tuned model is a complete artefact. And optimisation risk scales informally: with a small task dataset, an unrestricted update has enough freedom to overfit the task format and disturb unrelated capabilities (catastrophic forgetting). PEFT methods attack all three by restricting the update to a small, structured parameter set while the base model stays frozen. *In Sentinel-LLM, this cost accounting is made concrete: results.json records trainable parameter counts for every model variant, so the efficiency claim is arithmetic, not rhetoric.*

2.2 The low-rank hypothesis

LoRA rests on an empirical observation about adaptation: although a pretrained weight matrix is full-rank, the *change* needed to specialise a model for a downstream task appears to have low intrinsic dimensionality -- the update dW that full fine-tuning would learn can be approximated well by a matrix of much lower rank. If dW has rank at most r, it can be factored as dW = B A, where A has shape (r, d_in) and B has shape (d_out, r). The factored form needs r x d_in + d_out x r = r (d_in + d_out) parameters instead of d_in x d_out. For a square 256 x 256 layer at r=8, that is 4,096 parameters instead of 65,536 -- about 6%. The approximation is a hypothesis, not a theorem: it holds well for task specialisation (style, format, domain classification, terminology) and less well when the task requires injecting large amounts of genuinely new knowledge, which is one reason retrieval augmentation and fine-tuning are complementary rather than competing techniques. Chapter 7 returns to this point, because the sweep in this project is precisely an empirical probe of how much rank the task actually needs -- and the answer, on this dataset, is "very little."

2.3 The LoRA update rule

For a frozen pretrained weight W_0 and an input vector x, a LoRA-adapted linear layer computes:

```
h = W_0 x + (alpha / r) * B A x
```

where A is (r, d_in), B is (d_out, r), r is the rank, and alpha is a constant scaling hyperparameter. Three design details matter, and all three are implemented explicitly in lora.py:

1. **B is initialised to zero; A is randomly initialised.** At the first training step, B A = 0, hence dW = 0 and the adapted layer computes exactly what the base layer computed. Training therefore starts from the base model's behaviour and departs from it gradually -- there is no initial "shock" to the network. This property is not assumed in this project; the self-test at the bottom of lora.py asserts it numerically (the adapted output equals the base output to within 1e-6 at initialisation). A is initialised with Kaiming uniform initialisation, so the product B A can grow in a well-scaled way once B moves away from zero.
2. **The scaling factor alpha / r.** The update is multiplied by alpha/r. This makes the *effective* step size of the adapter roughly independent of rank: without it, changing r would silently change the magnitude of the update and confound rank comparisons with learning-rate comparisons. In this project alpha is always set to twice the rank (alpha = 2r) in the sweep -- 8/4, 16/8, 32/16 -- so the scaling factor is a constant 2.0 across the three rank configurations, and any difference between them is attributable to rank itself (and, in the fourth configuration, to learning rate).
3. **Dropout on the adapter path.** A small dropout (0.05 in this project) is applied to the input of the low-rank branch during training as light regularisation of the adapter; it is disabled at evaluation.

The trainable parameter count for one adapted layer is r (d_in + d_out). For this project's two target layers at the selected rank r=16: fc1 (128 -> 256) contributes 16 x (128 + 256) = 6,144 parameters, and fc2 (256 -> 27) contributes 16 x (256 + 27) = 4,528 parameters, totalling 10,672 -- exactly the trainable_lora_params recorded in results.json. The reader can verify every sweep row the same way: at r=8 the total is 8 x 384 + 8 x 283 = 5,336; at r=4 it is 2,668.

2.4 Freezing, merging and multi-adapter serving

Because W_0 is frozen, a LoRA deployment stores one base model plus one small adapter per specialisation. Two serving patterns follow. **Merging**: since the update is linear, it can be folded back into the base weight, W = W_0 + (alpha / r) B A, producing a single ordinary weight matrix with zero additional inference latency; lora.py exposes this as the merged_weight() method. **Swapping**: alternatively, adapters can be kept separate and hot-swapped -- one base model in memory, many cheap task adapters (in this project the entire adapter is 10,672 floating-point numbers; for a real LLM, adapters are typically a few tens of megabytes against gigabytes for the base). This is the operational pattern that makes per-task, per-team or per-tenant specialisation practical, and it is the pattern the project's roadmap points to: one base LLM, one Sentinel adapter per SOC function, with a router in front. Merging is preferable when a single specialisation is deployed permanently; swapping is preferable when several specialisations share one serving process.

2.5 QLoRA: quantised base, full-precision adapters

QLoRA (Dettmers et al., 2023) pushes the memory argument one step further: if the base model is frozen anyway, it does not need to be stored at full precision during training. QLoRA stores the base weights in 4-bit **NF4** (NormalFloat4), a quantisation format whose levels are chosen to match the approximately normal distribution of neural network weights, and applies **double quantisation** -- the quantisation constants themselves are quantised -- to squeeze out further memory. Computation is performed by dequantising to a 16-bit compute dtype (bfloat16 here) on the fly, while the LoRA adapters are trained in full precision. Two further techniques from the QLoRA paper matter in practice: gradient checkpointing to trade compute for activation memory, and paged optimisers to survive memory spikes. The net effect, demonstrated in the original paper at 65B scale on a single GPU, is that LLM fine-tuning moves from datacentre hardware to a single consumer or free-tier cloud GPU. Sentinel-LLM's train_lora_peft.py follows this recipe exactly: BitsAndBytesConfig with load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_use_double_quant=True, bnb_4bit_compute_dtype=bfloat16, and prepare_model_for_kbit_training before the PEFT configuration is attached. Chapter 4.8 reads that script line by line.

2.6 Instruction tuning and prompt conditioning

Both halves of Sentinel-LLM train on the same format: an *instruction*, an *input*, and a target *output*. This is instruction tuning -- the model learns not merely "what label goes with this log line" but "what to do when asked, in this way, about this input." The local classifier operationalises this literally: its input encoding is encode(instruction + " " + input), so the instruction tokens are part of the model's input on every example. The production script does the same thing in the idiom of chat LLMs, rendering each example through the model's chat template with a system prompt ("You are Sentinel, a cybersecurity analyst assistant..."), a user turn containing instruction plus input, and an assistant turn containing the output.

A consequence worth understanding -- and deliberately surfaced by demo.py -- is **prompt conditioning**: because the instruction is part of the input, the same text under a different instruction is a different input and can receive a different prediction. demo.py therefore performs *instruction routing*: it scores a submitted text under all six canonical task instructions and reports the best-fitting frame. The project README records that using the wrong instruction measurably degrades predictions (a phishing email scored under the log-triage instruction is liable to be classified as a log threat). This mirrors the behaviour of production LLMs, where the framing of a prompt is part of the model's effective input, and it is why the demo is a more honest demonstration of instruction tuning than a bare classifier call would be.

2.7 The security domain: six tasks and one shared vocabulary

The six tasks (Chapter 5 gives the full specification) were chosen to span the first-pass judgements of Section 1.1: log_triage, mitre_mapping, phishing_detection, ioc_extraction, cve_explanation and incident_severity. Three domain structures deserve note. **MITRE ATT&CK** is the industry's shared knowledge base of adversary tactics and techniques; mapping behaviour to technique IDs (here T1110 Brute Force, T1046 Network Service Discovery, T1071 Application Layer Protocol, T1190 Exploit Public-Facing Application, T1566 Phishing, T1059 Command and Scripting Interpreter) is what makes an observation interoperable with detection rules, threat reports and other teams' work. **IOCs** are the concrete artefacts -- IP addresses, domains, file hashes, URLs -- that can be blocked, searched and hunted across an estate; extraction is the bridge from prose intelligence to machine action. **Severity rubrics** (here a four-level P1-P4 scale, labelled sev_critical, sev_high, sev_medium, sev_low) convert an incident narrative into a queue priority, and consistency of that conversion is exactly what fine-tuning is for. Modern open "cybersecurity LLM" efforts -- the family of CyberLLM / SecurityLLM-style projects and security-tuned Llama/Qwen variants -- follow the same template: take a general instruction model, instruction-tune it on SOC-shaped tasks (log analysis, threat-intel extraction, ATT&CK mapping, vulnerability explanation), and evaluate on held-out security tasks. Sentinel-LLM is a small, transparent instance of that template, with its evaluation protocol exposed rather than asserted.

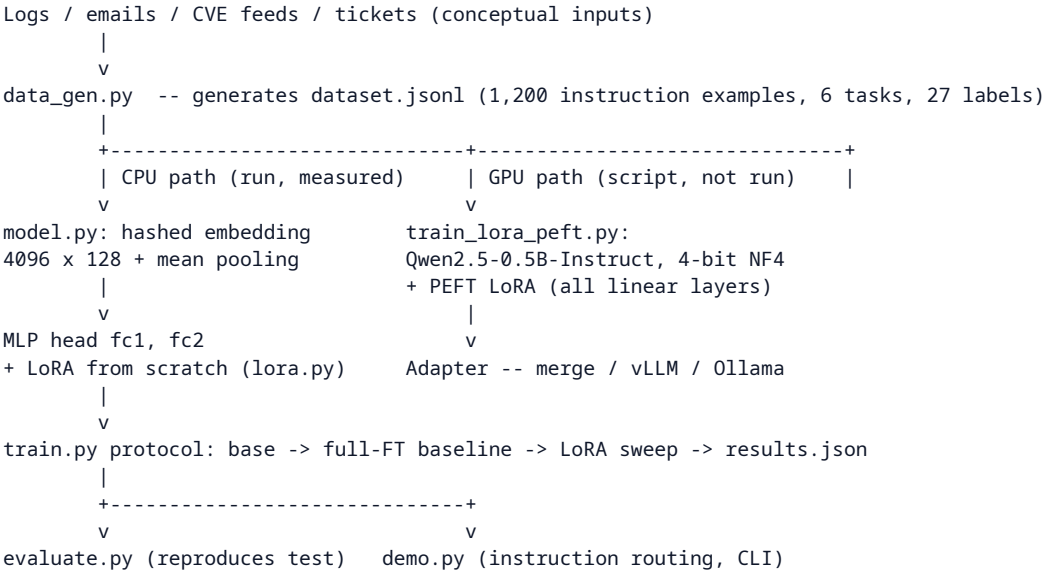
2.8 Measuring classification quality

The project's models are classifiers over a unified 27-label space, so the primary metric is **accuracy**: the fraction of examples whose predicted label equals the true label. Three refinements make the accuracy numbers interpretable. **Chance level**: with 27 labels, uniform random guessing scores 1/27 = 3.70%; the untrained model scores 3.66% on test, confirming both that the pipeline is wired correctly (an untrained net should sit at chance) and that later gains are learned, not artefacts. **Per-task accuracy**: the 27 labels partition by task, so accuracy can be reported per task; this is what reveals that the base model's weakness is concentrated in three tasks rather than spread evenly. **Validation loss as a secondary signal**: cross-entropy loss on the validation split measures not just whether the argmax is right but how confidently and calibratedly the model distributes probability; when accuracies tie, loss separates configurations -- which is precisely what happens in this project's sweep, and why the selection rule (best validation accuracy, tie-break lower validation loss, then fewer trainable parameters) is stated in results.json before it is applied. Finally, the split discipline matters as much as the metric: hyperparameters are selected on validation, and each finished model is evaluated on test once, so the test numbers are not silently fitted by the sweep.

3. System Design and Architecture

3.1 Two halves, one dataset

Sentinel-LLM is deliberately designed as two training paths fed by a single dataset, so that the methodology can be *verified cheaply* and *executed for real* on appropriate hardware:



The design principle is *one source of truth at each level*: dataset.jsonl is the single dataset artefact consumed by both paths; train.py's split logic is the single definition of train/validation/test (evaluate.py reproduces it exactly and asserts that the checkpoint's label ordering matches); results.json is the single source for every reported number.

3.2 Module map

File	Lines	Responsibility
data_gen.py	241	Generate dataset.jsonl: six task generators, randomised entities, seed 42
lora.py	114	LoRA from scratch: LoRALinear, inject_lora, freeze/count utilities, self-test
model.py	66	Hashed tokenisation/encoding + SentinelClassifier (embedding, mean pool, MLP)
train.py	284	Full protocol: splits, Stage A base, Stage B full-FT, Stage C sweep, Stage D test, checkpoint + results.json
evaluate.py	68	Independent re-evaluation of the shipped checkpoint on the reproduced test split
demo.py	100	CLI inference with instruction routing over six task frames
train_lora_peft.py	128	Production QLoRA fine-tune of Qwen2.5-0.5B-Instruct (GPU only)
requirements.txt	--	CPU path dependencies: torch, numpy, pandas
requirements_gpu.txt	--	GPU path dependencies: transformers, peft, trl, datasets, bitsandbytes, accelerate, sentencepiece, protobuf

3.3 Data flow in the CPU path

An example flows through the CPU path as follows. data_gen.py emits a JSON object with instruction, input, output, task and label. train.py groups examples by label, shuffles each group with a dedicated random generator (seed 123), and assigns the first 70% to train, the next 15% to validation and the remainder to test -- a stratified split, so every label appears in every split in proportion. Text is converted to token ids once per split (the Split class pre-encodes) by model.py's encode(): lowercase, extract tokens with a regular expression that keeps security-relevant punctuation (dots, slashes, colons, hashes, hyphens -- so that IP addresses, URLs and CVE IDs survive as tokens), map each token through an MD5 hash modulo 4096, and pad per batch with a mask. The classifier embeds ids (4096 x 128 table), mean-pools over unmasked positions, and passes the pooled vector through fc1 (128 -> 256, ReLU, dropout 0.15), then fc2 (256 -> 27) to produce logits. During Stages B and C the starting weights are a deep copy of the Stage-A checkpoint; in Stage C, inject_lora replaces fc1 and fc2 with LoRALinear wrappers and unfreeze_lora_only leaves only the A/B matrices trainable. Training uses AdamW with cosine annealing and cross-entropy loss in batches of 32. Evaluation computes overall accuracy, mean loss and per-task accuracy in batches of 64 under no_grad.

3.4 Design decisions and their reasons

Hashed embeddings instead of a learned vocabulary or a pretrained encoder. The local model's job is to prove the LoRA pipeline, not to win a language-modelling contest. Hashing (MD5, stable across processes -- deliberately *not* Python's salted hash()) gives a fixed 4,096-token vocabulary with no vocabulary-building pass, no external files and full determinism. The known cost is collisions: distinct tokens can share an embedding. At this scale that cost is acceptable and is disclosed; the production path uses the LLM's real tokenizer.

Mean pooling instead of sequence modelling. A pooled bag-of-embeddings classifier is the smallest architecture that can exhibit the phenomena under study (adaptation, rank, forgetting-free freezing). It is also brutally honest: it cannot exploit word order, so its successes and failures are attributable to lexical signal and to the training protocol, which is what the study measures.

LoRA targets the MLP layers (fc1, fc2). In a transformer, LoRA is typically applied to attention and MLP projections. In this two-layer MLP classifier, fc1 and fc2 are the *only* weight matrices besides the embedding table, so they are the direct analogue. The embedding table (524,288 of the base model's 564,251 parameters) is part of the frozen base in the LoRA stages -- a deliberate illustration that in real LLMs, too, the bulk of parameters sit outside the adapter.

A shared base checkpoint for every comparison. Stages B and C both start from the Stage-A weights. Any difference between full fine-tuning and LoRA is therefore a difference of method, not of initialisation or of data order luck.

Selection on validation, with the tie-break written down. The rule -- best validation accuracy; tie-break lower validation loss; then fewer trainable parameters -- is recorded in results.json as sweep_selection. As Chapter 7 shows, the tie-break is not decorative: all four configurations tie on validation accuracy, so the rule, not discretion, picks the winner.

4. Implementation -- A File-by-File, Function-by-Function Walkthrough

This chapter explains every file in the project, function by function, with the key code and the reasoning behind it. Line counts are from the shipped sources.

4.1 data_gen.py -- the dataset generator (241 lines)

The generator builds 200 examples per task (1,200 total) with random.seed(42) fixed at import, and shuffles the combined list before writing dataset.jsonl, one JSON object per line.

Entity helpers. rand_ip(private=False) fabricates IP addresses, choosing public-looking first octets from a fixed list (45, 91, 103, 185, 193, 203, 198, 77, 89) or 10.x private addresses. rand_domain(malicious=False) builds plausible phishing domains from lure words ("secure-verify", "login-update", "account-confirm") with low-reputation TLDs (xyz, top, click), or ordinary corporate domains otherwise. rand_hash() returns a genuine SHA-256 hex digest of a random string -- correctly formatted IOC material. rand_port() samples from a list of operationally interesting ports (22, 80, 443, 3389, 445, 3306, and others). Constants USERS, HOSTS and SERVICES supply usernames, hostnames and daemon names. The point of the randomisation is methodological: IPs, users, counts, ports and hashes differ across examples, so a model cannot score by memorising one canonical string per class; it must use the distributional pattern (for example, "many failed passwords from one source" versus "one accepted password").

gen_log_triage(). Picks one of five labels (benign, brute_force, port_scan, malware_beacon, sql_injection) and renders a log line from label-specific templates: accepted-password and HTTP-200 lines for benign; repeated-failure lines with a randomised attempt count (15-200) for brute_force; firewall/IDS lines listing 5-9 sampled ports for port_scan; fixed-interval (30/60/120/300 s) DNS or proxy callbacks to a malicious domain for malware_beacon; and web lines carrying one of seven real SQL-injection payloads ("' OR '1'='1' --", UNION SELECT forms, SLEEP-based probes) for sql_injection. Each output is a short analyst-style verdict naming the class and the recommended action -- the same text the production LLM would be trained to generate.

gen_mitre(). Draws from a six-row MITRE table pairing technique IDs (T1110, T1046, T1071, T1190, T1566, T1059) with names and behaviour descriptions, wraps the description in one of four phrasings ("Observed behaviour: ...", "Incident note: ...", and so on), and targets the technique ID as the label. Note that the label is the *code*, which forces the model to learn the code-to-behaviour association rather than outputting a descriptive phrase -- the hard part of the real task, in miniature.

gen_phishing(). Phishing examples combine urgency, suspicious domains, credential or payment requests and lure language; legitimate examples are routine business mail (meeting notes, order shipping, newsletters, HR notices) from ordinary domains. The two classes are balanced within the task's 200 examples by uniform label sampling.

gen_ioc(). Builds a short intelligence report containing exactly one IOC type (ip_ioc, domain_ioc, hash_ioc, url_ioc) or all four (multiple_ioc), with the label naming the type(s) present. This makes the task a *typing* problem, not merely a spotting problem: the model must say what kind of indicator the text contains.

gen_cve(). Uses a five-row CVE table (rce -- the Log4Shell pattern; sqli; xss; priv_esc -- the Sudo "Baron Samedit" pattern; info_disclosure -- path traversal) and pairs each input with a

plain-language explanation and mitigation dictionary. The label is the vulnerability class.

gen_severity(). Renders incident narratives at four severity levels, from ransomware with confirmed exfiltration (sev_critical) down to a forgotten password (sev_low), each with a P1-P4 justification in the output.

main(per_task=200). Runs all six generators, shuffles, writes the JSONL, and prints per-task and per-label counts (from Python's Counter) as a build-time sanity check.

4.2 lora.py -- LoRA from scratch (114 lines)

This is the project's technical core. It has four public components.

class LoRALinear(nn.Module). The constructor takes an existing nn.Linear, the rank r (validated > 0), alpha and dropout. It stores the layer as self.base and **freezes it** (requires_grad=False for every base parameter). It then creates the two adapter parameters: lora_A of shape (r, in_features), initialised with Kaiming uniform, and lora_B of shape (out_features, r), initialised to **zeros** -- the line on which the whole method's starting behaviour depends. It precomputes self.scaling = alpha / r and an nn.Dropout (or nn.Identity when dropout is 0). The forward method computes base(x) plus (dropout(x) @ A^T @ B^T) *scaling: the input is projected down to rank r by A, back up to the output dimension by B, scaled, and added to the frozen base output. merged_weight() returns base.weight + scaling* (B @ A) under no_grad -- the folded weight used when an adapter is merged for deployment (Section 2.4).

inject_lora(model, target_suffixes=("fc1", "fc2"), r, alpha, dropout). Walks the model's module tree recursively; whenever a child's name matches a target suffix and the child is an nn.Linear, it replaces that attribute on the parent with a LoRALinear wrapping the original layer, recording the full dotted name. It returns the list of replaced names -- which train.py stores in results.json as lora_config.target_layers (["fc1", "fc2"]), so the report of what was adapted is generated by the code, not typed by hand.

freeze_all(model) / unfreeze_lora_only(model). freeze_all switches every parameter off. unfreeze_lora_only first freezes everything, then re-enables gradients only on lora_A and lora_B of every LoRALinear module, and returns the count of trainable adapter parameters it found -- a built-in audit of the freezing step.

count_parameters(model). Returns total parameters, trainable parameters, and the trainable percentage -- the function that produces the efficiency numbers used throughout this report (for example 574,923 total and 10,672 trainable in the selected model).

Self-test (the __main__ block). Builds a 128 -> 256 linear layer, wraps it at r=8, checks the output shape (4, 256) on a random batch, prints parameter statistics, and asserts the zero-init property: the LoRA layer's output equals the unwrapped layer's output to within 1e-6. Running python lora.py is therefore a one-command proof of the implementation's defining invariant.

4.3 model.py -- tokenisation and the classifier (66 lines)

tokenize(text) lowercases and extracts tokens with the regular expression [a-z0-9_.\-/:/#]+ -- note the deliberate inclusion of dots, slashes, colons and hashes, which keeps IPs, URLs, file paths and CVE IDs intact as single-ish tokens rather than shattering them into digits. **hash_token(token, vocab_size)** maps a token to int(md5(token), 16) mod 4096; the docstring-equivalent comment in the code notes why MD5 is used: it is stable across runs and processes, unlike Python's per-process salted hash(). **encode(text)** returns the id list (substituting a single "<empty>" token for empty input). **encode_batch(texts)** pads a batch to its maximum length and returns the index tensor plus a float mask.

class SentinelClassifier. Constants: VOCAB_SIZE=4096, EMBED_DIM=128, HIDDEN_DIM=256. The module holds an nn.Embedding(4096, 128, padding_idx=0), fc1 = Linear(128, 256), ReLU, Dropout(0.15), and fc2 = Linear(256, num_labels). Its parameter total can be verified by hand: embedding 4096 x 128 = 524,288; fc1 32,768 weights + 256 bias; fc2 6,912 weights + 27 bias; total 564,251 -- the base_total_params in results.json. forward(indices, mask) embeds, multiplies by the mask, mean-pools over time (dividing by the clamped mask sum), applies fc1/ReLU/dropout, and returns fc2 logits.

4.4 train.py -- the experimental protocol (284 lines)

load_splits(). Reads dataset.jsonl, builds the sorted 27-label list and label2id map, groups rows by label, and -- with a dedicated random.Random(123) -- shuffles each label group and cuts it at 70% and 85%, producing 829 train, 180 validation and 191 test rows. Stratification by label (rather than a global shuffle) guarantees that even infrequent labels are represented in all three splits.

class Split. Pre-encodes every row's text as encode(instruction + " " + input) -- the instruction-conditioning decision of Section 2.6 lives on this line -- and stores targets and task names. Its batch(idxs) method collates padded index/mask/target tensors on demand.

evaluate(model, split). Runs inference in batches of 64, accumulates summed cross-entropy, overall accuracy and per-task correct/total counters, and returns accuracy, mean loss and the per-task dictionary. This single function produces every accuracy and loss figure in results.json, for every model variant -- there is no second evaluation implementation to drift out of sync.

train_epochs(model, split, epochs, lr, seed, verbose). AdamW (weight decay 0.01) over *only the parameters with requires_grad=True* -- which is what makes the same function correct for full training (Stage A/B) and for adapter-only training (Stage C) without modification. A CosineAnnealingLR scheduler spans the epoch budget; batches of 32 are reshuffled each epoch; per-epoch mean loss is printed when verbose.

fresh_from(base_state, num_labels). Constructs a new classifier and loads a deep copy of the Stage-A state dict -- the mechanism enforcing the shared-starting-point design.

main() -- the four stages. Stage A: seed 42, build the classifier, record the *untrained* test accuracy (0.0366), train 1 epoch at lr 3e-3 (47.3 s), record base test 0.8325 and validation 0.8278, snapshot the weights. Stage B: fresh copy, 8 epochs at lr 3e-3 (217.6 s), record full-FT test 1.0000, validation 0.9944, 564,251 trainable. Stage C: for each of the four sweep configurations (seed 7 for comparability), fresh copy, inject_lora on fc1/fc2 with the configuration's r/alpha and dropout 0.05, unfreeze_lora_only, train 10 epochs at the configuration's learning rate (verbose off), evaluate on validation, and keep the best under the sort key (val_accuracy, -val_loss, -trainable_params). Stage D: evaluate the winner once on test (1.0000), save the checkpoint (state dict, labels, label2id, and the

full config including LoRA r/alpha) to sentinel_lora.pt, and write results.json -- including the sweep table, the full-FT baseline block, the efficiency summary string (generated from the measured counts), timing, device ("CPU") and the honesty note about synthetic data and the un-run GPU script. Total recorded wall time for the whole protocol: 1,308.9 s on the shared VM used for the build (the README notes the run was concurrent with other builds; a single epoch takes roughly 3 s on an idle CPU).

4.5 evaluate.py -- independent verification (68 lines)

evaluate.py exists so that the headline number does not rest on the training script's say-so. **load_model()** reads sentinel_lora.pt, rebuilds the classifier with the checkpoint's stored dimensions, re-injects LoRA with the checkpoint's stored r and alpha (dropout 0.0), loads the state dict and sets eval mode. **main()** *reproduces* the test split from dataset.jsonl using the identical stratified procedure and seed 123, asserts that the checkpoint's label list equals the dataset's sorted labels, scores all 191 test rows in batches of 64, and prints overall accuracy (191/191, 1.0000), per-task accuracy, and -- if there were any errors -- the top confusions (true -> predicted). For the shipped checkpoint it prints "No misclassifications on the test split." A result that can be re-derived by a separate short script is a result a reviewer can trust.

4.6 demo.py -- inference with instruction routing (100 lines)

TASK_INSTRUCTIONS holds the six canonical instruction strings, identical to those used in training. **load_model()** mirrors evaluate.py's checkpoint reconstruction. **predict_routed(model, labels, text)** is the interesting function: for each task, it encodes that task's instruction plus the user text, runs the model, takes the softmax top-3, and records the best candidate; across tasks it returns the candidate with the highest confidence -- task frame, predicted label, confidence and alternatives. **show()** prints a fixed-format block (input, routed task, predicted class, confidence as a percentage, top-3 list). **main()** supports three modes: piped stdin (python demo.py -), a free-text argument, or six built-in SAMPLES when called bare. The verified outputs recorded in the project README illustrate the routing working as intended: a brute-force log line is routed to log_triage and classified brute_force at 99.92% confidence; a phishing email to phishing at 100%; a ransomware narrative to sev_critical at 100%; Log4Shell text to rce at 99.93%; an IOC report to ip_ioc at 99.95%.

4.7 requirements files

requirements.txt pins the CPU path to torch (>= 2.0), numpy and pandas -- nothing else -- which is why the entire study reproduces on a laptop or a free CPU instance. requirements_gpu.txt lists the production stack: torch >= 2.3, transformers >= 4.44, peft >= 0.12, trl >= 0.10, datasets >= 2.20, bitsandbytes >= 0.43, accelerate >= 0.33, sentencepiece and protobuf.

4.8 train_lora_peft.py -- the production QLoRA script, line by line (128 lines)

This script has **not been run** (it requires a CUDA GPU, and it refuses to proceed without one). It is included because it is the real deliverable for a SOC deployment, and it deserves the same scrutiny as the code that did run. The walkthrough follows the file top to bottom.

Header and argument parsing. The module docstring states the target model (Qwen/Qwen2.5-0.5B-Instruct by default), the method (QLoRA: 4-bit NF4 base, LoRA r=16, alpha=32 on q/k/v/o/gate/up/down projections), where to run it (Kaggle/Colab T4/P100), and what happens afterwards (adapter in <output_dir>/adapter, merging via merge_and_unload, serving via vLLM).

parse_args() exposes --model, --data (default dataset.jsonl -- the same file as the CPU path), --output_dir, --epochs (3.0), --lr (2e-4 -- note the order-of-magnitude difference from the toy classifier's 8e-3: real LLM fine-tunes use far smaller steps), --batch_size (8), --grad_accum (4, for an effective batch of 32), --lora_r (16), --lora_alpha (32), --max_len (512) and --seed (42).

format_example(ex, tokenizer). Converts one dataset row into chat format: a system message fixing the analyst persona ("You are Sentinel, a cybersecurity analyst assistant. Answer precisely and cite the threat type, MITRE technique or IOCs where relevant."), a user message of instruction + blank line + input, and an assistant message containing the target output. It renders the three messages with tokenizer.apply_chat_template and returns {"text": ...} -- the field TRL's SFTTrainer consumes. This function is the exact point where the CPU study's (instruction + input) conditioning becomes a real LLM's chat conditioning; the dataset does not change at all.

main() -- imports and the GPU guard. All heavy imports (torch, datasets, transformers, peft, trl) are deferred inside main() so the file *parses* on a CPU-only machine -- a small, thoughtful touch that lets linters and reviewers read it anywhere. The first real statement raises SystemExit if torch.cuda.is_available() is false, with a message pointing back to train.py for CPU use. A GPU script that fails loudly on CPU is safer than one that appears to start and then dies mid-load.

Data loading. Rows are read from the JSONL, formatted, wrapped in a Hugging Face Dataset, and split with train_test_split(test_size=0.15, seed=args.seed) into train/evaluation portions for the SFT run. (Note the honest difference from the CPU protocol: this split is a simple 85/15 shuffle for training-time evaluation, not the stratified 70/15/15 study split -- appropriate for a production training script, and one reason its future results must be reported separately, not conflated with results.json.)

Tokeniser setup. AutoTokenizer is loaded for the base model; if pad_token is None it is set to eos_token, the standard fix that batched causal-LM training requires.

Quantisation configuration. BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4", bnb_4bit_compute_dtype=torch.bfloat16, bnb_4bit_use_double_quant=True) is the QLoRA recipe of Section 2.5 verbatim: NF4 storage, double-quantised constants, bf16 compute. AutoModelForCausalLM.from_pretrained loads the base under this config with device_map="auto", and prepare_model_for_kbit_training performs the required preparation (including enabling gradient flow through the quantised base to the adapters).

PEFT configuration. LoraConfig(r=16, lora_alpha=32, lora_dropout=0.05, bias="none", task_type="CAUSAL_LM", target_modules=[q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]) -- LoRA on **all** linear projections, attention and MLP alike, following current PEFT practice rather than the older attention-only convention. The alpha = 2r relationship matches the local sweep's convention, so the configuration that the CPU study selected (r=16, alpha=32) is the configuration the production script ships by default. model.print_trainable_parameters() is called so the run log itself records the trainable fraction -- for a model of roughly half a billion parameters, r=16 on all linear layers trains on the order of 1% of parameters, and the resulting adapter is a few tens of megabytes. Those are scale expectations stated in the project README; they are **not** measured results, and no accuracy figure of any kind is attached to this script anywhere in the project.

Training configuration and run. SFTConfig sets the output directory, 3 epochs, per-device batch 8 with gradient accumulation 4, learning rate 2e-4 with a cosine schedule and 3% warmup, logging every 10 steps, saving per epoch, bf16 enabled, no external reporting (report_to=[]), seed 42 and max_length 512. SFTTrainer is constructed with the model, config, train/eval datasets, the PEFT

config and the tokenizer as processing_class; trainer.train() runs the fine-tune; the adapter and tokenizer are saved to <output_dir>/adapter, and the script prints the two follow-on options: merge with PeftModel.merge_and_unload(), or serve base + adapter (for example with vLLM). Evaluation of the resulting LLM -- generation on a held-out split with exact-match on labels/techniques and IOC precision/recall -- is specified in the README roadmap and remains **future work with no claimed numbers**.

---

5. Dataset

5.1 Schema and generation

The dataset is a single JSONL file, dataset.jsonl, in which every line is an object with five fields: instruction (the task framing), input (the artefact to judge), output (the analyst-style target response), task (one of six task names) and label (the classification target, one of 27 values). This is the standard instruction-tuning schema, chosen so that the identical file feeds both the local classifier (which trains on the label) and the GPU LLM script (which trains on the output text). Generation is by data_gen.py with generator seed 42 and 200 examples per task.

5.2 The six tasks and 27 labels

Task	Labels (count)	What the model must learn
log_triage	benign, brute_force, port_scan, malware_beacon, sql_injection (5)	Distinguish routine log lines from four attack patterns
mitre_mapping	T1110, T1046, T1071, T1190, T1566, T1059 (6)	Bind behaviour descriptions to ATT&CK technique codes
phishing_detection	phishing, legitimate (2)	Separate social-engineering lures from business mail
ioc_extraction	ip_ioc, domain_ioc, hash_ioc, url_ioc, multiple_ioc (5)	Identify which IOC type(s) a report contains
cve_explanation	rce, sqli, xss, priv_esc, info_disclosure (5)	Classify a vulnerability description by class
incident_severity	sev_critical, sev_high, sev_medium, sev_low (4)	Apply a four-level severity rubric to a narrative

Total: 6 tasks, 27 labels, 1,200 examples (200 per task before splitting).

5.3 Splits

train.py's load_splits() performs a label-stratified split (seed 123) at 70% / 15% / 15%, yielding 829 training, 180 validation and 191 test examples (the integers differ slightly from exact percentages because each label group is cut independently and rounded). Stratification guarantees every one of the 27 labels is represented in every split. The roles are strict: training data fits parameters; validation data selects hyperparameters (Chapter 6); test data is evaluated once per finished model.

5.4 What the dataset is -- and is not

The dataset is **synthetic and templated**. Each class is rendered from a handful of templates with randomised slots (IPs, users, hosts, ports, counts, domains, hashes). This has three consequences, stated here and analysed in Chapter 8. First, the classes carry strong lexical signals -- the word patterns separating "beaconing every 60s to a DGA domain" from "backup completed successfully" are consistent by construction -- so very high accuracy is attainable and 100% test accuracy, while a genuine measurement, is a measurement *on this distribution*. Second, randomisation of the slots prevents pure string memorisation from explaining the results: the model must use pattern-level features. Third, the dataset cannot measure robustness to paraphrase, obfuscation, adversarial evasion or class imbalance, none of which it contains. It is a *pipeline-and-method* dataset: exactly what is needed to verify a from-scratch LoRA implementation and to compare adaptation methods fairly, and explicitly not a benchmark of SOC readiness. Public corpora named in the roadmap (NVD CVE descriptions, phishing corpora, IDS log datasets) are the correct next data step.

6. Experiments

6.1 Experimental questions

The protocol was designed to answer four questions:

Q1 (sanity). Does an untrained model sit at chance, and how far does a brief 1-epoch base training get?

Q2 (efficiency). Can LoRA, training under 2% of parameters, match a full fine-tuning baseline that trains 100%?

Q3 (hyperparameters). How do LoRA rank (r = 4, 8, 16) and learning rate (8e-3 vs 3e-3) affect validation accuracy and validation loss?

Q4 (localisation). On which tasks does adaptation actually help?

6.2 Protocol

All runs share: the dataset and splits of Chapter 5; the SentinelClassifier architecture (vocab 4096, embedding dim 128, hidden dim 256, dropout 0.15 in the base head); cross-entropy loss; AdamW (weight decay 0.01); cosine learning-rate annealing; batch size 32; CPU execution. Stage A trains all parameters for 1 epoch at lr 3e-3 (the "brief base", deliberately undertrained to stand in for a general pretrained model that has not been specialised). Stage B continues from Stage A's weights, training all 564,251 parameters for 8 epochs at lr 3e-3 (full fine-tuning baseline). Stage C, for each sweep configuration, starts again from Stage A's weights, injects LoRA into fc1 and fc2 (dropout 0.05 on the adapter path), freezes everything else, and trains only the adapter for 10 epochs. Stage D evaluates the validation-selected LoRA model on test once.

6.3 Sweep design

Four configurations were swept, with alpha always equal to 2r so that the alpha/r scaling is a constant 2.0 in the three rank arms:

#	r	alpha	lr	Trainable params	Isolates
1	4	8	8e-3	2,668 (0.47%)	Smallest adapter
2	8	16	8e-3	5,336 (0.94%)	Rank, mid
3	16	32	8e-3	10,672 (1.86%)	Rank, high
4	8	16	3e-3	5,336 (0.94%)	Learning rate at fixed rank

Selection rule (recorded in results.json): best validation accuracy; ties broken by lower validation loss, then by fewer trainable parameters. Seeds: base training seed 42; all sweep runs seed 7 (same seed across configurations, so data order is held constant and differences are attributable to the configuration). Timing was recorded per stage on the build VM (CPU, shared with concurrent builds -- timings are therefore indicative, and no efficiency claim in this report rests on wall-clock).

6.4 Baselines

Three reference points bracket the LoRA results: the **untrained** model (chance-level check), the **brief base** (what you get with minimal general training and no specialisation stage), and **full fine-tuning** (the expensive method LoRA must match). Per-task accuracies are recorded for the base, for full FT and for the selected LoRA model.

7. Results and Analysis

Every number in this chapter is copied from results.json (Appendix A). Accuracies are fractions in the source file and are reported here as percentages.

7.1 Headline results

Model	Trainable params	Val accuracy	Test accuracy
Untrained classifier	--	--	3.66%
Brief base (1 epoch, all params)	564,251 (100%)	82.78%	83.25%
Full fine-tuning (8 epochs, all params)	564,251 (100%)	99.44%	100.00%
LoRA, selected config (r=16, alpha=32, lr 8e-3, 10 epochs)	10,672 (1.86%)	99.44%	100.00%

Reading the table. The untrained model scores 3.66% against a 27-label chance level of 3.70% -- the pipeline passes its sanity check (Q1). One epoch of general training lifts the model to 83.25%: the task is largely learnable from lexical patterns, but a substantial residue of error remains. Both serious adaptation methods close that residue completely on test: full fine-tuning reaches 100.00%, and LoRA reaches 100.00%. The difference between them is not accuracy but cost: 564,251 trainable parameters against 10,672 -- LoRA's total model is 574,923 parameters (the 564,251 base parameters plus the 10,672 adapter parameters), of which 1.86% ever receive a gradient. The recorded efficiency_summary states it exactly: "LoRA (best config) trains 10672 of 574923 params (1.86%) vs full FT 564251 (100%) -- test 1.0000 (LoRA) vs 1.0000 (full FT)." The ratio 564,251 / 10,672 = 52.9 is where the "52.9x fewer" headline comes from. Against the brief base, LoRA's gain is +16.75 percentage points (accuracy_gain_vs_base = 0.1675). Q2 is therefore answered in the affirmative *on this dataset*: parity with full fine-tuning at 1.86% of the trainable parameters.

7.2 The sweep: rank buys confidence, not accuracy

r	alpha	lr	Trainable	Val accuracy	Val loss	Train time (s)
4	8	8e-3	2,668 (0.47%)	99.44%	0.0475	256.8
8	16	8e-3	5,336 (0.94%)	99.44%	0.0230	228.5
16	32	8e-3	10,672 (1.86%)	99.44%	0.0065	270.9
8	16	3e-3	5,336 (0.94%)	99.44%	0.0426	275.5

This is the most instructive table in the project, precisely because it refuses to tell a simple "bigger rank wins" story. All four configurations classify 179 of the 180 validation examples correctly (99.44%) -- one validation example is missed by every configuration. Accuracy, the metric most reports would headline, is *saturated* and cannot distinguish the configurations at all. The distinguishing signal is validation loss, which behaves beautifully systematically:

> **Rank effect (lr fixed at 8e-3):** loss falls monotonically as rank grows -- 0.0475 (r=4) -> 0.0230 (r=8) -> 0.0065 (r=16). The r=16 adapter is not more *often* right than the r=4 adapter; it is more *decisively* right, assigning higher probability to the correct class. In operational terms, its confidence scores are more trustworthy -- exactly the property that matters if a SOC pipeline uses confidence thresholds to route low-confidence cases to humans. Rank, on this dataset, buys calibration headroom.

Learning-rate effect (rank fixed at 8): lr 8e-3 reaches loss 0.0230 against 0.0426 at lr 3e-3 -- the lower rate underfits within the 10-epoch budget. Configuration 4 exists to isolate this effect, and it does: same rank, same parameter count, nearly double the loss.

Selection: because accuracy ties, the recorded tie-break (lower validation loss, then fewer parameters) selects r=16, alpha=32, lr 8e-3 -- the lowest loss, at the price of the largest adapter. The rule also exposes the honest counterfactual: had the tie-break preferred parameter count over loss, r=4 (0.47% of parameters, same validation accuracy) would have been selected, and the project README names r=4 "the practical pick if adapter size matters most." Both readings are defensible; the protocol makes the choice explicit and reproducible rather than post hoc.

The deeper lesson for a learner is about the low-rank hypothesis itself (Section 2.2). A rank-4 update to two small matrices -- 2,668 numbers -- already contains enough capacity to express essentially all of the task adaptation this dataset demands, starting from a 1-epoch base. That is the intrinsic-dimensionality claim made measurable. It also predicts, correctly, where the method would strain: a task whose adaptation requires substantially new *knowledge* (rather than new *routing* of existing features) would push the required rank up, and the saturation pattern would break. Chapter 8 returns to this.

7.3 Per-task analysis: where adaptation earns its cost

Task	Brief base	LoRA (selected)	Full FT
log_triage	71.88%	100.00%	100.00%
mitre_mapping	62.50%	100.00%	100.00%
cve_explanation	66.67%	100.00%	100.00%
phishing_detection	100.00%	100.00%	100.00%
ioc_extraction	100.00%	100.00%	100.00%
incident_severity	100.00%	100.00%	100.00%

The base model's errors are not spread thinly; they are concentrated in exactly three tasks, and the pattern is interpretable. **MITRE mapping (62.50%)** is the weakest: it is the only task whose labels are arbitrary codes. Nothing in general lexical statistics says that password-guessing behaviour should be called "T1110" rather than "T1046"; the code-binding is a pure convention that must be *taught*, which is what adaptation is for -- and indeed LoRA teaches it completely (+37.50 points, the largest gain in the study). **CVE explanation (66.67%)** requires binding vulnerability vocabulary to class conventions (is a path-traversal flaw info_disclosure or something else?), a similar convention-learning problem (+33.33 points). **Log triage (71.88%)** mixes both difficulties: several attack classes share surface vocabulary (an IP, a port, a daemon name appear in benign and malicious lines alike), so the decision boundary depends on pattern combinations that one epoch does not settle (+28.12 points). Conversely, phishing_detection, ioc_extraction and incident_severity are already at 100.00% after one epoch: their classes are separated by strong, unambiguous lexical cues (urgency-plus-credential-request language; the mere formatted presence of a hash; words like "ransomware" and "exfiltration" versus "forgot password"). For those tasks, additional adaptation -- LoRA or full -- adds nothing measurable on this data.

This localisation is the practically useful output of the study (Q4). A practitioner with limited compute should fine-tune for the tasks where the base model demonstrably fails -- convention-binding and boundary-refinement tasks -- and should not spend budget "improving" tasks that are already saturated. It also warns against reading the aggregate 83.25% base accuracy as uniform mediocrity:

it is three solved tasks averaged with three unsolved ones.

7.4 Error analysis

Three error facts survive the final models, and all are visible in the artefacts rather than hidden. First, **one validation example is misclassified by every swept LoRA configuration and by the full fine-tuning baseline** (all sit at 179/180): a single stubborn example that the shared Stage-A representation apparently places on the wrong side of every learned boundary -- a reminder that some residual errors are properties of the data/representation, not of the adaptation method. Second, the **test set is fully solved by both adaptation methods** (191/191 for the LoRA checkpoint, independently reproduced by evaluate.py), so no test confusion matrix exists to analyse; evaluate.py's confusion report is empty by fact, not by omission. Third, the project README records a qualitative failure mode outside the test distribution: texts framed under the *wrong instruction* can be misclassified (prompt conditioning, Section 2.6), and free-form paraphrases far outside the templates are the expected failure frontier. A serious error analysis on real SOC data would start exactly there.

7.5 Timing and cost accounting

Recorded times (CPU, shared build VM, concurrent with other builds -- indicative only): Stage A base training 47.3 s; full fine-tuning stage 217.6 s; the selected LoRA configuration 270.9 s; individual sweep arms between 228.5 s and 275.5 s; the complete protocol (base + baseline + four-arm sweep + evaluation + checkpointing) 1,308.9 s wall-clock. Two honest observations follow. LoRA is *not* faster per epoch here -- it trains the same number of epochs over the same data through the same frozen forward pass, and on a tiny CPU model the adapter arithmetic is not the bottleneck. LoRA's savings are in **trainable-parameter memory, optimiser state, checkpoint size and multi-task storage**, and (at LLM scale) in making training fit on smaller GPUs at all -- not in a wall-clock promise at toy scale. Any report claiming LoRA is "faster" without this qualification should be read sceptically; this one does not claim it.

7.6 What the results do not show

The 100.00% figures are real measurements on the defined test split, and they are also -- by the dataset's construction (Chapter 5.4) -- measurements on a strongly separable synthetic distribution. They establish: the from-scratch LoRA implementation is correct; the protocol is sound; LoRA matches full fine-tuning here at 1.86% of the trainable parameters; rank and learning rate affect confidence as analysed; adaptation gains localise to convention-bound tasks. They do **not** establish accuracy on production logs, robustness to paraphrase or evasion, performance under class imbalance, or anything at all about the QLoRA LLM, which has not been run. Chapter 8 develops these limits; the distinction between "the pipeline is proven" and "the problem is solved" is the single most important sentence in this report.

8. Discussion, Limitations and the Production Path

8.1 Interpreting the efficiency result

The headline comparison -- 10,672 versus 564,251 trainable parameters at identical test accuracy -- replicates, at miniature scale, the central empirical claim of the LoRA literature: for task specialisation, the adaptation lives in a low-dimensional subspace, and paying for a full-rank update buys nothing the task can use. Three properties make this replication credible rather than anecdotal. The comparison is *controlled* (same starting checkpoint, same data, same evaluation code). The baseline is *strong* (full fine-tuning is itself trained to saturation, not crippled to make LoRA look good -- it too reaches 100.00%). And the sweep shows the result is *not a knife-edge*: every rank from 4 to 16 saturates validation accuracy, so the parity finding does not depend on having luckily chosen r=16. Where the miniature differs from LLM practice is scale of the base: here the frozen embedding table dominates the parameter count, so the *trainable fraction* (1.86%) is in the same range as typical LLM LoRA setups (on the order of 1%), even though the absolute model is five orders of magnitude smaller than the production target.

8.2 Limitations and threats to validity

1. **Synthetic, templated data (the dominant limitation).** Classes are generated from templates with strong lexical cues. Real SOC text is noisy, misspelled, multilingual, deliberately obfuscated by adversaries, and brutally imbalanced (benign events outnumber attacks by orders of magnitude). Accuracy on this dataset must not be quoted as SOC performance. The numbers validate method and pipeline; a deployment claim would require the roadmap's real corpora and a re-run of the identical protocol.
2. **A classifier is not a generative LLM.** The local model predicts one of 27 labels. It cannot produce the free-text explanations, novel IOC spans or severity justifications that the dataset's output field contains and that the production LLM is trained to generate. Generative quality (exact-match on techniques, IOC precision/recall, explanation faithfulness) is unevaluated -- because the generative model has not been run.
3. **The GPU run is unexecuted.** train_lora_peft.py is reviewed code, not measured science. No accuracy, loss or qualitative sample is claimed for Qwen2.5-0.5B anywhere in this project, and the first roadmap item is to run it on Kaggle/Colab and publish its held-out evaluation alongside -- not instead of -- this report's numbers.
4. **Bag-of-embeddings representation.** Mean pooling discards word order and hashing admits collisions; some real distinctions (negation, sequence, "failed then succeeded" versus "succeeded then failed") are invisible to the local model by construction. Its failures would be representation failures first.
5. **Single-seed sweep.** All sweep arms share seed 7 and the Stage-A checkpoint derives from seed 42; the study does not estimate seed variance. The saturation pattern makes the qualitative conclusions robust, but exact loss values (0.0065 and friends) are single-run measurements.
6. **Timing environment.** Wall-clock figures were recorded on a shared VM running concurrent builds; they are reported for scale, not as benchmarks.
7. **Closed label world.** The classifier must choose one of 27 labels; it has no "unknown / none of these" option. A production triage system needs calibrated abstention (route low-confidence

cases to analysts) -- noted in the roadmap, and made more plausible by the sweep's finding that higher rank improves confidence quality.

8.3 From study to service: the production path

The path from this project to a working SOC assistant is explicit, and each step reuses an artefact that already exists. **Step 1:** run train_lora_peft.py on a Kaggle/Colab GPU (requirements_gpu.txt) against the same dataset.jsonl; evaluate generation on a held-out split with exact-match for labels/techniques and span-level precision/recall for IOCs; publish the adapter and the eval table. **Step 2:** replace or augment the synthetic data with public corpora -- NVD CVE descriptions for cve_explanation, public phishing corpora for phishing_detection, IDS/auth log datasets for log_triage -- and re-run both the CPU protocol (as a cheap regression check) and the GPU fine-tune. **Step 3:** serve one base LLM with per-task adapters and a router -- the production analogue of demo.py's instruction routing (Section 4.6) -- merging the adapter (merged_weight() logic, PeftModel.merge_and_unload() in the PEFT world) only if a single-task deployment is preferred. **Step 4:** add calibration and abstention, streaming ingestion, and analyst-in-the-loop feedback so that corrected predictions become new training rows. The reason this roadmap is credible is that none of its steps requires redesigning the dataset schema, the task definitions or the evaluation discipline -- those are the parts this project actually built and verified.

8.4 Relation to current practice

Sentinel-LLM sits squarely inside the current state of the art for domain adaptation of LLMs, at teaching scale. LoRA (Hu et al., 2022) is the reference PEFT method; QLoRA (Dettmers et al., 2023) is the reference memory recipe (NF4, double quantisation, bf16 compute) and is implemented faithfully in the GPU script. The convention alpha = 2r and LoRA on all linear projections reflects current PEFT practice. Instruction tuning on SOC-shaped tasks mirrors the CyberLLM / SecurityLLM family of efforts that specialise open LLMs for security work. And the evaluation discipline -- held-out validation for selection, a full fine-tuning baseline, per-task breakdowns, an independent evaluation script -- is the protocol a reviewer of any of those systems would demand. What this project adds to that landscape is not a new method but a transparent, fully reproducible worked example in which every mechanism can be inspected in 1,001 lines of code.

9. Conclusion and Future Work

Sentinel-LLM set out to implement LoRA from scratch, apply it to cybersecurity threat intelligence, and measure -- honestly -- what parameter-efficient fine-tuning does and does not buy. The implementation is verified by its own self-test (zero-initialised B makes a fresh adapter an exact no-op) and by an independent evaluation script that reproduces the headline result. The controlled study answers its four questions: an untrained model sits at chance (3.66%); LoRA matches full fine-tuning exactly (100.00% test) with 10,672 trainable parameters against 564,251 (1.86%, 52.9x fewer); the hyperparameter sweep shows accuracy saturating across ranks while validation loss falls monotonically with rank (0.0475 to 0.0065) and responds clearly to learning rate -- rank buys confidence; and the per-task analysis shows adaptation gains concentrating where conventions must be taught (MITRE mapping +37.50 points, CVE explanation +33.33, log triage +28.12) and vanishing where lexical cues already suffice. The limits are equally clear: synthetic separable data, a classifier rather than a generator, and a production QLoRA script that is written and reviewed but not yet run.

Future work, in priority order: (1) execute the Kaggle QLoRA run and publish its held-out generative evaluation; (2) re-run the identical protocol on public real-world corpora and report the (certainly lower, genuinely informative) numbers; (3) add confidence-based abstention and analyst routing, exploiting the calibration behaviour the sweep exposed; (4) build the multi-adapter serving demo (one base, per-task adapters, router) as the deployable counterpart of demo.py; (5) extend the label space (more ATT&CK techniques, more CVE classes) and test whether the rank-saturation pattern survives a harder, less separable task -- the experiment that would most deepen the science in this report.

10. Learning Guide

10.1 Concept checklist

A reader who can explain each item below, in their own words, understands this project:

- [ ] Full fine-tuning: what is updated, and the three costs that scale with trainable parameters
- [ ] The low-rank hypothesis: why dW = B A can suffice for specialisation, and when it should fail
- [ ] The LoRA forward rule h = W_0 x + (alpha/r) B A x, with every symbol's shape
- [ ] Why B is zero-initialised and A is random -- and how to *prove* a fresh adapter is a no-op
- [ ] The role of alpha/r scaling, and why this project fixes alpha = 2r in the sweep
- [ ] Parameter arithmetic: trainable count r (d_in + d_out) per layer; verify 10,672 for this model
- [ ] Freezing and gradient flow: why AdamW in train.py only steps the adapter in Stage C
- [ ] Merging versus swapping adapters, and when each is the right serving pattern
- [ ] QLoRA: NF4, double quantisation, bf16 compute, and why the base can be 4-bit but adapters cannot
- [ ] Instruction tuning and prompt conditioning, including demo.py's instruction routing
- [ ] Stratified splitting, validation-only model selection, and the tie-break rule
- [ ] Accuracy saturation versus loss: why loss separated configurations that accuracy could not
- [ ] Per-task error localisation as a guide to where fine-tuning budget belongs
- [ ] The six SOC tasks, the MITRE ATT&CK mapping idea, and IOC types
- [ ] Why 100% on synthetic data validates a pipeline but does not certify SOC performance

10.2 Interview questions with model answers

Q1. What is LoRA, in one minute? LoRA is a parameter-efficient fine-tuning method. Instead of updating a pretrained weight matrix W_0, you freeze it and learn a low-rank update dW = B A, where A projects the input down to a small rank r and B projects back up. The layer computes h = W_0 x + (alpha/r) B A x. Only A and B are trained -- typically around 1% of the parameters -- yet the adapted model routinely matches full fine-tuning, because task specialisation turns out to need only a low-rank change. In Sentinel-LLM, the selected adapter trains 10,672 of 574,923 parameters (1.86%) and matches the full fine-tuning baseline's test accuracy exactly.

Q2. Why is B initialised to zero? What breaks if both A and B are random? Zero B makes B A = 0 at initialisation, so the adapted model starts out behaving *exactly* like the base model and departs gradually as training proceeds. If both matrices were random, the adapter would inject a random perturbation at step zero -- the model would start from a damaged version of the base, and early training would waste itself undoing that damage. The property is checkable: lora.py's self-test asserts the adapted output equals the base output at initialisation.

Q3. How many parameters does a LoRA adapter add, and how do you compute it? For a layer of shape (d_out, d_in) at rank r: A has r x d_in and B has d_out x r, so the adapter has r (d_in + d_out) parameters, versus d_in x d_out for the full weight. In this project at r=16: fc1 (128 -> 256) contributes 16 x 384 = 6,144, fc2 (256 -> 27) contributes 16 x 283 = 4,528, totalling 10,672 -- which is exactly what the code's parameter counter reports.

Q4. What is the alpha/r scaling for? Why did the sweep use alpha = 2r? The update is multiplied by alpha/r so that changing rank does not silently change the update's magnitude -- without it, a rank comparison would be confounded with an effective-learning-rate comparison. Setting alpha = 2r in every sweep arm makes the scaling a constant 2.0, so differences between arms are attributable to rank (or, in the fourth arm, to learning rate) alone.

Q5. Your sweep shows every rank at 99.44% validation accuracy. Was the sweep useless? No -- it produced the study's most useful finding. Accuracy had saturated, but validation *loss* separated the configurations cleanly: 0.0475 at r=4, 0.0230 at r=8, 0.0065 at r=16. Higher rank buys confidence/calibration, not correctness, on this data. The pre-registered tie-break (best accuracy, then lowest loss, then fewest parameters) selected r=16; and the sweep also showed that even r=4 at 0.47% of parameters reaches accuracy parity, which is the practical choice if adapter size dominates. A sweep that reveals *which metric still carries signal* is a successful sweep.

Q6. Why compare against full fine-tuning at all? Because "LoRA gets 100%" is meaningless without knowing what the expensive method gets. The full-FT baseline starts from the same checkpoint, trains all 564,251 parameters to saturation and also reaches 100.00% test. LoRA's claim is parity at 1.86% of the trainable parameters -- a claim that only exists relative to that baseline. Weakening or omitting the baseline is the most common way PEFT results are oversold.

Q7. LoRA matched full FT here. When would it not? When the adaptation requires substantial new knowledge or a high-rank change: teaching a model facts it has never seen, a very different output distribution, or many unrelated tasks in one adapter. The low-rank hypothesis is about *specialisation*. In this project the signal is visible in miniature: rank 4 already suffices because the task is mostly re-routing existing lexical features; a harder, less separable corpus is exactly the experiment proposed to find where saturation breaks.

Q8. What is QLoRA and what do NF4 and double quantisation do? QLoRA fine-tunes a model whose frozen base weights are stored in 4 bits. NF4 (NormalFloat4) is a 4-bit format whose levels are spaced for normally distributed weights, minimising quantisation error for the values networks actually have. Double quantisation quantises the quantisation constants themselves, saving additional memory. Computation dequantises to bf16 on the fly, while the LoRA adapters train in full precision. The effect is LLM fine-tuning on a single modest GPU -- which is why train_lora_peft.py can target Kaggle's free tier.

Q9. Why does demo.py "route" the input across instructions instead of just classifying once? Because the model was trained on (instruction + input) pairs -- instruction tuning makes the instruction part of the input. The same text scored under the wrong task's instruction is a different input and can be misclassified. demo.py therefore scores the text under all six canonical instructions and reports the best-fitting frame. It is a small, honest demonstration of prompt conditioning, the same phenomenon that makes prompt framing matter for production LLMs.

Q10. Your test accuracy is 100%. Should a CISO deploy this in a SOC? No -- and the report says so. The dataset is synthetic and templated, hence strongly separable; 100% validates the LoRA implementation, the protocol and the efficiency comparison, not performance on real logs, which are noisy, imbalanced and adversarial. Deployment requires the roadmap: real corpora (NVD descriptions, phishing datasets, real log data), a re-run of the same protocol, calibration with abstention to human analysts, and the generative QLoRA model actually trained and evaluated. Quoting 100% as SOC accuracy would be exactly the kind of claim this project is careful not to make.

Q11. What does "stratified split" mean here, and why does model selection use validation rather than test? Stratified means the 70/15/15 split is performed *within each label group*, so all 27

labels appear in train, validation and test in proportion (829/180/191 examples). Selection uses validation because choosing hyperparameters on the test set would fit the test set by selection and inflate the reported score; test is touched once per finished model, so the test numbers remain an honest estimate. evaluate.py independently reproduces the test split (same seed 123) to verify the checkpoint's score.

Q12. The base model was already at 100% on three tasks. What does that teach about fine-tuning strategy? That adaptation value is task-local, not global. Phishing detection, IOC extraction and severity already had unambiguous lexical separation, so one epoch solved them and further training added nothing measurable. MITRE mapping (62.50%), CVE explanation (66.67%) and log triage (71.88%) needed convention-learning and boundary refinement, and that is where LoRA's gains concentrated (+37.50, +33.33 and +28.12 points). The strategy lesson: profile per-task baselines first, and spend fine-tuning budget where the base demonstrably fails.

Q13. How would you serve several LoRA specialisations in production? Keep one copy of the base model in memory and swap small adapters per task or per tenant -- adapters are megabytes, base models are gigabytes. A router (the production version of demo.py's instruction routing) picks the adapter per request. Alternatively, if one specialisation is permanent, merge the adapter into the base weights (W = W_0 + (alpha/r) B A, the merged_weight() operation) for zero added latency and a single artefact.

10.3 Exercises to extend the project

1. **Break the saturation.** Add paraphrase augmentation (rewrite templates with synonyms, reorder clauses) to data_gen.py and re-run train.py. Report at which point the sweep's accuracy stops saturating and rank starts to matter for accuracy.
2. **Rank the embedding.** Apply LoRA-style low-rank updates to a projection of the embedding table, or simply unfreeze the embedding in Stage C, and measure the change in the trainable fraction and validation loss.
3. **Seed variance.** Re-run the sweep with three different Stage-C seeds and report mean and standard deviation of validation loss per configuration. Does the rank ordering survive?
4. **Calibration and abstention.** Using the selected checkpoint's softmax confidences, choose a threshold that routes the least-confident 5% of test inputs to a "human review" bucket. Report the accuracy on the retained 95% and the characteristics of the routed examples.
5. **Run the real thing.** Execute train_lora_peft.py on Kaggle, evaluate generation on a held-out split (exact-match for MITRE codes and labels; precision/recall for IOC spans), and write the one-page addendum this report deliberately leaves blank.

11. How to Run

Local CPU path (a few minutes on a normal laptop; the recorded full protocol took 1,308.9 s on the shared build VM):

```
pip install -r requirements.txt        # torch, numpy, pandas only
python data_gen.py                     # -> dataset.jsonl (1,200 examples, seed 42)
python lora.py                         # LoRA self-test: shapes + zero-init no-op check
python train.py                        # Stage A base -> Stage B full-FT -> Stage C sweep
                                       # -> results.json + sentinel_lora.pt
python evaluate.py                     # independent check: 191/191 on the test split
python demo.py                         # six built-in samples with instruction routing
python demo.py "Failed password for root from 91.2.3.4, attempt 87 of 200"
```

GPU production path (Kaggle / Colab, T4/P100; **not run for this report**):

```
pip install -r requirements_gpu.txt    # transformers, peft, trl, datasets, bitsandbytes,
accelerate
python data_gen.py                     # if dataset.jsonl is not present
python train_lora_peft.py --epochs 3 --output_dir ./sentinel-qlora-out
# Adapter lands in ./sentinel-qlora-out/adapter
# Merge: PeftModel.from_pretrained(base, adapter).merge_and_unload()
# Serve: base + adapter via vLLM / Ollama
```

12. References

1. Hu, E. J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." International Conference on Learning Representations (ICLR), 2022. arXiv:2106.09685.

2. Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L. "QLoRA: Efficient Finetuning of Quantized LLMs." Advances in Neural Information Processing Systems (NeurIPS), 2023. arXiv:2305.14314.

3. MITRE ATT&CK Knowledge Base. MITRE Corporation. Technique IDs T1059, T1046, T1071, T1110, T1190, T1566 as used in the mitre_mapping task.

4. National Vulnerability Database (NVD), National Institute of Standards and Technology -- CVE description conventions referenced by the cve_explanation task.

5. Qwen Team. Qwen2.5-0.5B-Instruct model card, Hugging Face -- production fine-tuning target in train_lora_peft.py.

6. Hugging Face PEFT, TRL and bitsandbytes documentation -- libraries used by the GPU script (versions in requirements_gpu.txt).

7. The CyberLLM / SecurityLLM family of open efforts on instruction-tuning LLMs for security operations tasks (log analysis, threat-intel extraction, ATT&CK mapping) -- representative of current domain-adaptation practice discussed in Sections 2.7 and 8.4.

Appendix A -- Full results.json


```
{
  "project": "Sentinel-LLM",
  "author": "Ashutosh Sahoo, M.Tech AI, NIT Silchar",
  "dataset_size": 1200,
  "train_size": 829,
  "val_size": 180,
  "test_size": 191,
  "num_labels": 27,
  "labels": [
    "T1046",
    "T1059",
    "T1071",
    "T1110",
    "T1190",
    "T1566",
    "benign",
    "brute_force",
    "domain_ioc",
    "hash_ioc",
    "info_disclosure",
    "ip_ioc",
    "legitimate",
    "malware_beacon",
    "multiple_ioc",
    "phishing",
    "port_scan",
    "priv_esc",
    "rce",
    "sev_critical",
    "sev_high",
    "sev_low",
    "sev_medium",
    "sql_injection",
    "sqli",
    "url_ioc",
    "xss"
  ],
  "tasks": [
    "cve_explanation",
    "incident_severity",
    "ioc_extraction",
    "log_triage",
    "mitre_mapping",
    "phishing_detection"
  ],
  "split": "stratified 70/15/15 train/val/test, seed 123",
  "total_params": 574923,
  "base_total_params": 564251,
  "trainable_lora_params": 10672,
  "trainable_pct": 1.86,
  "lora_config": {
    "r": 16,
    "alpha": 32,
    "lr": 0.008,
    "target_layers": [
      "fc1",
      "fc2"
    ],
    "dropout": 0.05
  },
  "base_epochs": 1,
  "lora_epochs": 10,
  "untrained_accuracy": 0.0366,
  "base_accuracy": 0.8325,
  "base_val_accuracy": 0.8278,
  "lora_accuracy": 1.0,
  "lora_val_accuracy": 0.9944,
  "accuracy_gain_vs_base": 0.1675,
  "per_task_accuracy_base": {
```

```
    "cve_explanation": 0.6667,
    "incident_severity": 1.0,
    "ioc_extraction": 1.0,
    "log_triage": 0.7188,
    "mitre_mapping": 0.625,
    "phishing_detection": 1.0
  },
  "per_task_accuracy_lora": {
    "cve_explanation": 1.0,
    "incident_severity": 1.0,
    "ioc_extraction": 1.0,
    "log_triage": 1.0,
    "mitre_mapping": 1.0,
    "phishing_detection": 1.0
  },
  "hyperparameter_sweep": [
    {
      "r": 4,
      "alpha": 8,
      "lr": 0.008,
      "epochs": 10,
      "trainable_params": 2668,
      "trainable_pct": 0.47,
      "val_accuracy": 0.9944,
      "val_loss": 0.0475,
      "training_time_sec": 256.8
    },
    {
      "r": 8,
      "alpha": 16,
      "lr": 0.008,
      "epochs": 10,
      "trainable_params": 5336,
      "trainable_pct": 0.94,
      "val_accuracy": 0.9944,
      "val_loss": 0.023,
      "training_time_sec": 228.5
    },
    {
      "r": 16,
      "alpha": 32,
      "lr": 0.008,
      "epochs": 10,
      "trainable_params": 10672,
      "trainable_pct": 1.86,
      "val_accuracy": 0.9944,
      "val_loss": 0.0065,
      "training_time_sec": 270.9
    },
    {
      "r": 8,
      "alpha": 16,
      "lr": 0.003,
      "epochs": 10,
      "trainable_params": 5336,
      "trainable_pct": 0.94,
      "val_accuracy": 0.9944,
      "val_loss": 0.0426,
      "training_time_sec": 275.5
    }
  ],
  "sweep_selection": "best validation accuracy; tie-break lower val loss, then fewer trainable
params",
  "full_finetuning_baseline": {
    "epochs": 8,
    "lr": 0.003,
    "trainable_params": 564251,
    "trainable_pct": 100.0,
    "val_accuracy": 0.9944,
    "test_accuracy": 1.0,
    "per_task_accuracy": {
      "cve_explanation": 1.0,
      "incident_severity": 1.0,
```

```
      "ioc_extraction": 1.0,
      "log_triage": 1.0,
      "mitre_mapping": 1.0,
      "phishing_detection": 1.0
    },
    "training_time_sec": 217.6
  },
  "efficiency_summary": "LoRA (best config) trains 10672 of 574923 params (1.86%) vs full FT
564251 (100%) \u2014 test 1.0000 (LoRA) vs 1.0000 (full FT).",
  "training_time_sec": 1308.9,
  "base_training_time_sec": 47.3,
  "lora_training_time_sec": 270.9,
  "device": "CPU",
  "notes": "Local demo: small hashed-embedding classifier proving the from-scratch LoRA pipeline
 with a SOTA-style sweep + full-FT baseline. Synthetic templated data is strongly separable (see
 README limitations). Full LLM QLoRA run: train_lora_peft.py on GPU \u2014 its accuracy is NOT
claimed here because it has not been run."
}
```

Appendix B -- File Listing

File	Lines	Role
data_gen.py	241	source
demo.py	100	source
evaluate.py	68	source
lora.py	114	source
model.py	66	source
train.py	284	source
train_lora_peft.py	128	source
dataset.jsonl	1200 examples	Generated dataset (data_gen.py, seed 42)
sentinel_lora.pt	--	Selected LoRA checkpoint (r=16, alpha=32) from train.py Stage D
results.json	--	Machine-readable source of every number in this report
train_log.txt	--	Console log of the recorded training run
requirements.txt	--	CPU dependencies: torch, numpy, pandas
requirements_gpu.txt	--	GPU dependencies for train_lora_peft.py
README.md	--	Project README (summary of this report)

End of report. Sentinel-LLM -- Ashutosh Sahoo, M.Tech (AI), NIT Silchar, November 2026. All measured results are from the local CPU study recorded in results.json; the GPU QLoRA script (train_lora_peft.py) has not been run and no result is claimed for it.